

	Ano Letivo: 20252	Turno: Noturno	Local: A definir
	Curso: ENGENHARIA DE SOFTWARE		Turma: 5ª Fase
Disciplina: ENGENHARIA DE DADOS		Professor(a): JORGE LUIZ DA SILVA	

Python para Engenharia de Dados

VISÃO GERAL

Gerenciador de versões do Python	Pyenv
Gerenciador de pacotes, bibliotecas Python	PipX
Gerenciador de pacotes, bibliotecas e ambientes virtuais	Poetry
Gerenciador de pacotes, bibliotecas e ambientes virtuais	Uv
Versionamento de código	Git
Repositório de código fonte com controle de versão Git	Github
Git Ignore .gitignore	Ignr
Estrutura de pastas e arquivos para projetos Python	Padrão de projetos
Framework para geração de testes no Python	Pytest
Bibliotecas para padrões de código	Docstrings, Blue, Isort, pip-audit, ruff
Documentação web padrão Markdown	Mkdocs
Automatizador de tarefas e atalho para comandos complexos	Taskipy

PYENV

Pyenv é uma ferramenta usada para gerenciar várias versões do Python em um sistema operacional. Ele permite que você altere facilmente a versão global do Python utilizada em seu sistema, bem como a versão específica (local) de cada projeto. É utilizada para garantir a compatibilidade com vários projetos que possam exigir diferentes versões do Python.

O que o pyenv faz...

- Permite alterar a versão global do Python por usuário.
- Fornece suporte para versões Python por projeto.
- Permite substituir a versão do Python através de uma variável de ambiente.

LINK

<https://github.com/pyenv/pyenv>

PRINCIPAIS COMANDOS

pyenv	Lista todas as opções disponíveis para o comando pyenv (help).
pyenv --version	Verifica a versão do Pyenv instalado.
pyenv install --list	Mostra todas as versões do Python disponíveis para instalação.
pyenv install <versão>	Instala uma versão do Python. Ex.: pyenv install 3.12.1
pyenv versions	Lista todas as versões do Python instaladas.
pyenv global <versão>	Define uma versão do Python como global do ambiente. Ex.: pyenv global 3.12.1
pyenv local <versão>	Define uma versão do Python dentro de uma pasta específica. Ex.: pyenv local 3.12.1

PIPX

O pipx é uma ferramenta utilizada em Python para instalar e gerenciar pacotes Python globais de forma isolada em ambientes virtuais, permitindo que os pacotes sejam executados como programas independentes.

O que o pipx faz...

- Instalação global isolada.
- Gestão simplificada.
- Execução de pacotes como aplicativos.

LINK

<https://pipx.pypa.io/stable/>

PRINCIPAIS COMANDOS

pipx	Lista todas as opções disponíveis para o comando pipx (help).
pipx --version	Verifica a versão do pipx instalado.
pipx install <pacote>	Instala um pacote como global no sistema.
pipx list	Mostra os pacotes instalados como global.

POETRY

Poetry é uma ferramenta para gerenciamento de dependências e empacotamento em Python. Ele permite declarar as bibliotecas que um projeto depende e faz sua gestão (instalar/atualizar) usando apenas um arquivo chamado pyproject.toml.

O que o poetry faz...

- Instalação de bibliotecas.
- Gerenciamento de versões de bibliotecas.
- Gerenciador de ambiente virtual.
- Instalação e distribuição de pacotes.
- Gestão de ambiente de desenvolvimento e testes.

LINK

<https://python-poetry.org/>

COMO INSTALAR

1. Abrir o shell do WSL.
2. Executar o comando:

```
pipx install poetry
```

Fonte: <https://python-poetry.org/docs/#installing-with-pipx>

PRINCIPAIS COMANDOS

poetry	Lista todas as opções disponíveis para o comando poetry (help)
poetry --version	Verifica a versão do poetry instalado.
poetry new <nome do projeto>	Cria um novo projeto no caminho especificado (pasta nova).
poetry init	Cria um arquivo pyproject.toml básico no diretório atual.
poetry install	Instala as dependências do projeto baseadas no arquivo pyproject.toml.
poetry add <pacote>	Adiciona um novo pacote e faz a gestão de suas dependências.
poetry remove <pacote>	Remove um pacote e suas dependências sem gerar quebra de outro pacote instalado que possui a mesma dependência.
poetry show	Mostra as dependências instaladas.
poetry shell	Cria e loga em um shell dentro do ambiente virtual.
poetry env use <versao_python>	Define a versão do Python a ser usada pelo ambiente virtual do projeto. Ex. poetry env use 3.12.1

poetry run <arquivo python ou ferramenta>	Executa um arquivo python ou uma ferramenta através do poetry.
--	--

UV (ASTRAL)

UV é uma ferramenta de gerenciamento de dependências para Python, feita em Rust. Seu foco é ser extremamente rápida, segura e compatível com ferramentas existentes como pip, pip-tools, virtualenv e poetry.

O que o UV faz...

- Gerenciamento de dependências (como pip).
- Resolução de versões compatível com o Poetry.
- Instalação de pacotes muito mais rápida que pip.
- Criação e ativação de ambientes virtuais.
- Instalação determinística de pacotes (lockfiles).
- Substitui ferramentas como pip, pip-tools, poetry e virtualenv.

LINK

<https://github.com/astral-sh/uv>

COMO INSTALAR

3. Abrir o shell do WSL.
4. Executar o comando:

```
pipx install uv
```

Fonte: <https://github.com/astral-sh/uv#installation>

PRINCIPAIS COMANDOS

uv	Mostra o help com todos os comandos disponíveis.
uv venv	Cria um novo ambiente virtual.
uv pip install <pacote>	Instala pacotes como o pip, só que mais rápido.
uv add <pacote>	Instala pacotes do projeto no ambiente virtual criado.
uv remove <pacote>	Remove pacotes do projeto no ambiente virtual criado.
uv pip uninstall <pacote>	Remove pacotes do ambiente.
uv pip freeze	Lista os pacotes instalados no ambiente.
uv pip list	Mostra os pacotes e suas versões.
uv sync	Instala os pacotes de acordo com um arquivo requirements.txt ou pyproject.toml, garantindo consistência.
uv pip compile	Gera um arquivo requirements.txt fixando versões a partir de um pyproject.toml ou requirements.in.
uv run <arquivo python ou ferramenta>	Executa um arquivo python ou uma ferramenta através do uv.

GIT

O Git é um sistema de controle de versão distribuído usado para gerenciar projetos de software, mas com aplicações em diversas áreas. Ele oferece um conjunto de ferramentas para o desenvolvimento colaborativo e o rastreamento de alterações em arquivos ao longo do tempo.

O que o git faz...

- Sistema de colaboração.
- Rastreamento de alterações.

- Controle de versão distribuído.

LINK

<https://git-scm.com/>

PRINCIPAIS COMANDOS

git	Lista todas as opções disponíveis para o comando git (help)
git --version	Verifica a versão do git instalada.
git init	Inicializa um novo repositório.
git clone <url>	Faz uma cópia (clone) de um repositório existente.
git branch	Lista todas as branches no repositório.
git branch <nome da branch>	Cria uma Branch.
git checkout <nome da branch>	Mudar de uma branch para outra.
git status	Mostra o status do diretório
git commit -m "mensagem de commit"	Realiza um commit com a mensagem descrita.
git log	Mostra o histórico dos commits.
git push	Envia as atualizações (commits) para o repositório remoto.
git pull	Busca as alterações do repositório remoto e integra (atualiza) com a branch local.

GITHUB

O GitHub é uma plataforma de hospedagem de código-fonte baseada na web, que utiliza o sistema de controle de versão Git. É utilizado por desenvolvedores para hospedar, revisar e colaborar em projetos de software, além de oferecer ferramentas para gerenciamento de projetos e integração contínua.

O que o github faz...

- Hospedagem de repositórios Git.
- Colaboração e revisão de código.
- Gerenciamento de problemas (issues) e projetos.

LINK

<https://github.com/>

COMO CRIAR UMA CONTA:

1. Abrir o site: <https://github.com/>
2. Clicar no botão Sign Up e preencher os dados solicitados.

Fonte: <https://github.com/>

PRINCIPAIS COMANDOS

N/A

GIT IGNORE (.gitignore) - IGNR

O arquivo ".gitignore" é um arquivo de configuração usado pelo Git para especificar quais arquivos e diretórios devem ser ignorados durante o controle de versão.

O ignr.py (ou simplesmente ignr) é um utilitário de linha de comando baseado em Python utilizado para gerar o arquivo .gitignore baseado em modelos pré-existent.

O que o ignr faz...

- Cria arquivos .gitignore com base em modelos pré-existentis.

LINK

<https://github.com/Antrikshy/ignr.py>

COMO INSTALAR:

1. Abrir o shell no WSL.
2. Executar o comando:

```
pipx install ignr
```

Fonte: <https://github.com/Antrikshy/ignr.py>

PRINCIPAIS COMANDOS

ignr -h	Lista todas as opções disponíveis para o comando ignr (help)
ignr -l	Lista todos os templates disponíveis para o .gitignore no gitignore.io.
ignr -p python	Gera em tela o conteúdo do arquivo .gitignore para a linguagem python baseado no gitignore.io.
ignr -n python	Cria um arquivo .gitignore para a linguagem python baseado no gitignore.io no diretório atual.

ESTRUTURA DE PASTAS E ARQUIVOS PARA PROJETOS PYTHON

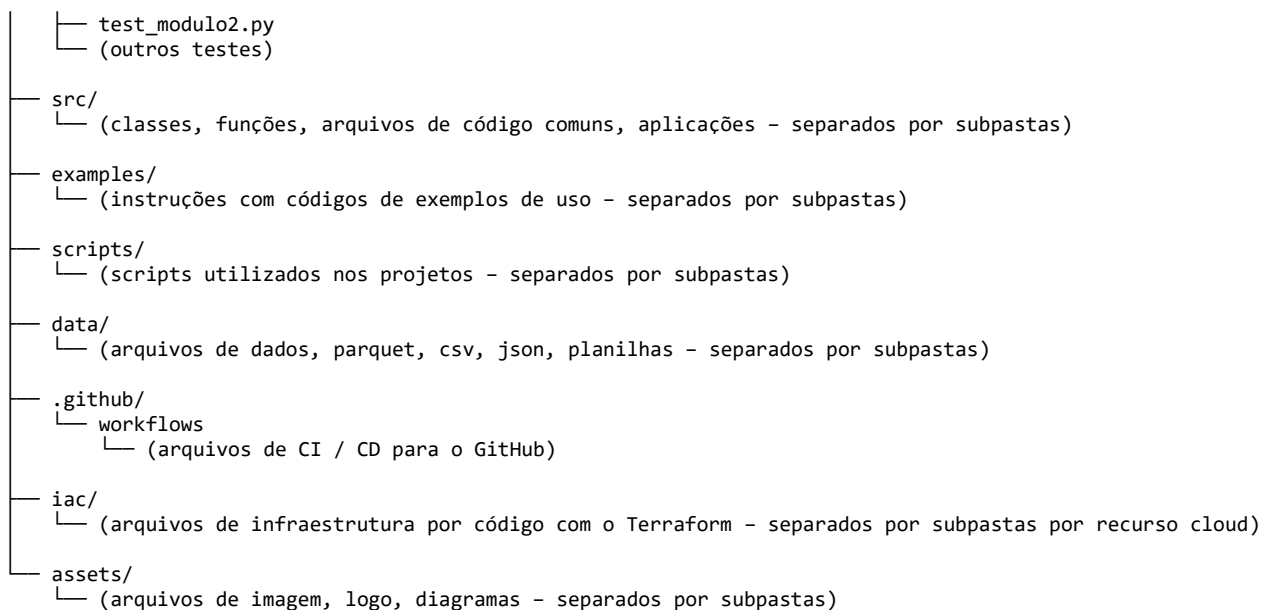
Uma estrutura de pastas para projetos Python geralmente segue convenções adotadas pela comunidade Python para organizar o código-fonte, testes, documentação de uma maneira padronizada.

O que a estrutura de pastas padronizada faz...

- Organização Lógica do Código.
- Separação de Responsabilidades.
- Documentação Centralizada.
- Facilita a Colaboração.

COMO CRIAR (SUGESTÃO):

```
meu-projeto/ (com traço "-" semântica, não precisar ter que pressionar shift para informar
              o nome do projeto)
├── README.md
├── LICENSE
├── .python-version
├── .gitignore
├── pyproject.toml
├── requirements.txt
├── requirements-dev.txt
├── (demais arquivos core do projeto)
├── logs/
│   └── (logs da aplicação ou projeto - podem ser separados por subpastas por assunto ou escopo)
├── docs/
│   └── (documentação do projeto - podem ser separados por subpastas por assunto ou escopo)
├── .venv/
│   └── (ambiente virtual do projeto)
├── meu_projeto/ (com sublinhado - padrão pacotes python)
│   ├── __init__.py
│   ├── main.py
│   ├── modulo1.py
│   ├── modulo2.py
│   └── (outros módulos)
├── tests/
│   └── test_modulo1.py
```



ARQUIVOS COMUNS

.gitignore	Arquivo que contém uma lista de arquivos e pastas que o Git ignora ao fazer commit.
README.md	Arquivo tipo markdown que fornece uma visão inicial e abrangente do projeto, além de fornecer instruções claras para outros desenvolvedores executarem o projeto com sucesso.
.python-version	Arquivo que especifica a versão do Python utilizada no projeto, normalmente gerado pelo Pyenv.
.github/workflows	Contém os scripts de workflow para GitHub Actions, permitindo a automação de pipelines CI/CD diretamente no GitHub.
pyproject.toml	Arquivo de configuração para ferramentas de construção de projetos Python, como Poetry ou PEP 517/518, contendo metadados do projeto e suas dependências.
requirements.txt	Uma lista de dependências do projeto para instalação com pip (obsoleto), o pyproject.toml atualmente é o mais comum.

SUGESTÃO PARA CRIAÇÃO DO ARQUIVO README

- **Nome do Projeto**
- **Descrição**
- **Disclaimer ou alertas/avisos**
- **Pré-requisitos:** Informações dos requisitos para usar o projeto (versão do Python, bibliotecas, dependências etc.)
- **Instalação:** Um roteiro com o passo-a-passo para reprodução da configuração do ambiente de dev.
- **Howto:** Exemplos de como utilizar o projeto, com imagens, códigos, etc.
- **Testes:** Como executar os testes criados no seu projeto.
- **Documentação:** Link para a documentação completa do seu projeto.

PYTEST

O Pytest é um framework de teste em Python utilizado para escrever e executar testes. Ele simplifica o processo de teste de software, fornecendo uma sintaxe limpa e poderosa, além de uma ampla gama de recursos para facilitar o desenvolvimento e a execução de testes.

O que o pytest faz...

- Testes unitários.
- Testes integrados.

- Testes funcionais.

LINK

<https://docs.pytest.org/en/8.2.x/contents.html>

COMO INSTALAR

1. Abrir o shell do WSL.
2. Executar o comando:

```
poetry add pytest
uv add pytest
```

Fonte: <https://docs.pytest.org/en/8.2.x/getting-started.html#install-pytest>

PRINCIPAIS COMANDOS

pytest --version	Mostra a versão atual do Pytest.
pytest -h	Lista todas as opções disponíveis para o comando pytest (help)
pytest	Executa todos os testes no diretório atual.
pytest <nome do arquivo>	Execute os testes do arquivo informado.
pytest -v	Habilita a saída com mais detalhes sobre os testes executados.
pytest -vv	Mostra mais detalhes que o -v.
pytest --html=report.html	Gera um relatório de testes em formato HTML (precisa do plugin pytest-html).

BIBLIOTECAS PARA PADRÃO DE CÓDIGO

pydocstyle	Valida a conformidade das docstrings com a convenção PEP257.	https://www.pydocstyle.org/en/stable/
isort	Organiza os imports de bibliotecas de forma ordenada e agrupada.	https://pycqa.github.io/isort/
black	Um formataador de código Python que valida e altera o código em conformidade a PEP 8.	https://black.readthedocs.io/en/stable/
blue	Uma outra opção para formatação de código Python que valida e altera o código em conformidade a PEP 8.	https://blue.readthedocs.io/en/latest/
pip-audit	Busca e corrige vulnerabilidades conhecidas em dependências instaladas.	https://github.com/pypa/pip-audit
pylint	Ferramenta de análise estática que verifica erros no código, impõe um padrão de codificação e sugere melhorias.	https://pylint.pycqa.org/en/latest/
ruff	Ferramenta extremamente rápida para linting e formatação de código Python, compatível com múltiplas ferramentas como flake8, isort e pycodestyle.	https://docs.astral.sh/ruff/

TASKIPY

O Taskipy é uma ferramenta de automação de tarefas em Python, que permite definir e executar tarefas repetitivas ou complexas por meio de scripts Python.

O que o taskipy faz...

- Cria uma espécie de alias para comandos completos e repetitivos.
- Testes integrados.
- Testes funcionais.

LINK

<https://github.com/taskipy/taskipy>

COMO INSTALAR

1. Abrir o shell do WSL.
2. Executar o comando:

```
poetry add taskipy
uv add taskipy
```

Fonte: <https://github.com/taskipy/taskipy?tab=readme-ov-file#installation>

COMO USAR COM POETRY

Como exemplo de uso, você pode alterar o arquivo **pyproject.toml** e incluir o bloco abaixo:

```
[tool.taskipy.tasks]
lint = "blue --check --diff . && blue --check --diff ."
test = "pytest --cov=nome_do_projeto -vv"
```

Ele cria dos “atalhos” para as linhas de comandos, simplificando a chamada para:

```
task lint #(é o mesmo que executar blue --check --diff . && blue --check --diff .)
task test #(é o mesmo que executar pytest --cov=nome_do_projeto -vv)
```

MKDOCS

O MKDocs é uma ferramenta utilizada para criar documentação estática a partir de arquivos Markdown.

O que o mkdocs faz...

- Geração de Documentação Estática.
- Simplicidade na Escrita (Markdown).
- Tema Personalizável.
- Integração com Git e GitHub.
- Permite o uso de extensões/plugins (CSS, diagramas etc).

LINK

<https://www.mkdocs.org/>

COMO INSTALAR

1. Abrir o shell do WSL.
2. Executar o comando:

```
poetry add mkdocs
uv add mkdocs
```

Fonte: <https://www.mkdocs.org/getting-started/>

PRINCIPAIS COMANDOS

mkdocs --version	Mostra a versão atual do mkdocs.
mkdocs	Lista todas as opções disponíveis para o comando mkdocs (help)
mkdocs new <nome do projeto>	Cria o arquivo mkdocs.yaml e uma pasta /docs com o arquivo index.md.
mkdocs serve	Visualiza localmente no formato web as páginas em construção.
mkdocs build	Constrói o site de documentação. Cria uma pasta chamada /site com os arquivos no formato web (html, css, js etc.)
mkdocs gh-deploy	Publica a documentação gerada pelo build no GitHub Pages (https://usuario.github.io/repositorio/).

